



AEU-6

AEU-6 INITIATED. LANE 6 — OPS, INTEGRATIONS & DOCS.

ObservabilityAgent · SREAgent · IntegrationAgent · DocsAgent — ALL FOUR IN PARALLEL.

ObservabilityAgent OUTPUT

app/middleware/observability.py

```
from __future__ import annotations

import time
from collections.abc import Awaitable, Callable

import structlog
from fastapi import Request, Response
from prometheus_client import (
    Counter, Gauge, Histogram, Info,
    generate_latest, CONTENT_TYPE_LATEST,
)
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.responses import PlainTextResponse

log = structlog.get_logger()

# — PROMETHEUS METRICS —————

REQUEST_COUNT = Counter(
    "lexcore_requests_total",
    "Total HTTP requests",
    ["method", "endpoint", "status_code", "tier"],
)

REQUEST_LATENCY = Histogram(
    "lexcore_request_latency_seconds",
    "HTTP request latency",
    ["endpoint"],
    buckets=[0.01, 0.05, 0.1, 0.2, 0.5, 1.0, 2.0, 5.0, 10.0],
)

SEARCH_LATENCY = Histogram(
    "lexcore_search_latency_seconds",
    "search_legal tool latency",
    buckets=[0.05, 0.1, 0.2, 0.4, 0.8, 1.5, 3.0],
)
```

```
RESEARCH_LATENCY = Histogram(  
    "lexcore_research_latency_seconds",  
    "research_task tool latency",  
    buckets=[0.5, 1.0, 2.0, 3.0, 5.0, 10.0, 20.0],  
)  
  
EMBED_LATENCY = Histogram(  
    "lexcore_embed_latency_seconds",  
    "Ollama embedding latency",  
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.0],  
)  
  
CACHE_HIT_COUNTER = Counter(  
    "lexcore_cache_hits_total",  
    "Cache hits per tool",  
    ["tool"],  
)  
  
CACHE_MISS_COUNTER = Counter(  
    "lexcore_cache_misses_total",  
    "Cache misses per tool",  
    ["tool"],  
)  
  
INGEST_JOBS = Counter(  
    "lexcore_ingest_jobs_total",  
    "Ingest jobs by status",  
    ["jurisdiction", "status"],  
)  
  
INGEST_DOCUMENTS = Counter(  
    "lexcore_ingest_documents_total",  
    "Documents ingested",  
    ["jurisdiction", "doc_type"],  
)  
  
DEAD_LETTER_DEPTH = Gauge(  
    "lexcore_dead_letter_depth",  
    "Number of tasks in dead-letter queue",  
    ["queue"],  
)  
  
ACTIVE_TENANTS = Gauge(  
    "lexcore_active_tenants",  
    "Active tenants by tier",  
    ["tier"],  
)  
  
QUERY_BUDGET_USED = Gauge(  
    "lexcore_query_budget_used_fraction",  
    "Fraction of monthly query budget used per tier",  
    ["tier"],  
)  
  
ERROR_RATE = Counter(  

```

```

        "lexcore_errors_total",
        "Application errors by type",
        ["error_type", "endpoint"],
    )

PRIVACY_VIOLATIONS = Counter(
    "lexcore_privacy_violations_total",
    "PII violations rejected at privacy guard",
    ["pattern_type"],
)

PII_FALSE_POSITIVE_RATE = Gauge(
    "lexcore_pii_false_positive_rate",
    "Estimated false positive rate from privacy guard",
)

CHUNK_QUALITY_REJECTED = Counter(
    "lexcore_chunk_quality_rejected_total",
    "Chunks rejected due to quality score < 0.7",
    ["jurisdiction"],
)

LEXCORE_INFO = Info(
    "lexcore",
    "LexCore service metadata",
)

# Set static service info at startup
LEXCORE_INFO.info({
    "version": "4.0.0",
    "schema": "4.0.0",
    "data_region": "northamerica-northeast1",
    "model": "mxbai-embed-large-v1",
})

class MetricsMiddleware(BaseHTTPMiddleware):
    """
    Records request count, latency, errors, cache hits per endpoint.
    Attaches tool-specific timers for search and research paths.
    """

    async def dispatch(
        self,
        request: Request,
        call_next: Callable[[Request], Awaitable[Response]],
    ) -> Response:
        start = time.monotonic()
        endpoint = self._normalize_path(request.url.path)
        tier = getattr(request.state, "tenant_tier", "unknown")

        try:
            response = await call_next(request)
            latency = time.monotonic() - start

            REQUEST_COUNT.labels(

```

```

        method=request.method,
        endpoint=endpoint,
        status_code=str(response.status_code),
        tier=tier,
    ).inc()

REQUEST_LATENCY.labels(endpoint=endpoint).observe(latency)

# Per-tool timers
if endpoint == "/api/v1/mcp/search":
    SEARCH_LATENCY.observe(latency)
    if getattr(request.state, "cache_hit", False):
        CACHE_HIT_COUNTER.labels(tool="search_legal").inc()
    else:
        CACHE_MISS_COUNTER.labels(tool="search_legal").inc()

elif endpoint == "/api/v1/mcp/research":
    RESEARCH_LATENCY.observe(latency)
    if getattr(request.state, "cache_hit", False):
        CACHE_HIT_COUNTER.labels(tool="research_task").inc()
    else:
        CACHE_MISS_COUNTER.labels(tool="research_task").inc()

if response.status_code >= 400:
    ERROR_RATE.labels(
        error_type=str(response.status_code),
        endpoint=endpoint,
    ).inc()

return response

except Exception as exc:
    latency = time.monotonic() - start
    ERROR_RATE.labels(error_type="500", endpoint=endpoint).inc()
    log.error(
        "lexcore.unhandled",
        endpoint=endpoint,
        latency_ms=int(latency * 1000),
        error=str(exc),
        exc_info=True,
    )
    raise

@staticmethod
def _normalize_path(path: str) -> str:
    """Collapse dynamic segments to prevent cardinality explosion."""
    segments = path.split("/")
    normalized = []
    for seg in segments:
        if len(seg) == 36 and seg.count("-") == 4:
            normalized.append("{uuid}")
        elif seg.startswith("lxc_") or len(seg) > 50:
            normalized.append("{token}")
        else:
            normalized.append(seg)
    return "/" + ".join(normalized)

```

```

def record_privacy_violation(pattern_type: str) -> None:
    PRIVACY_VIOLATIONS.labels(pattern_type=pattern_type).inc()

def record_chunk_rejected(jurisdiction: str) -> None:
    CHUNK_QUALITY_REJECTED.labels(jurisdiction=jurisdiction).inc()

def record_ingest_job(jurisdiction: str, status: str) -> None:
    INGEST_JOBS.labels(jurisdiction=jurisdiction, status=status).inc()

def record_ingest_document(jurisdiction: str, doc_type: str) -> None:
    INGEST_DOCUMENTS.labels(jurisdiction=jurisdiction, doc_type=doc_type).inc()

def record_embed_latency(latency_seconds: float) -> None:
    EMBED_LATENCY.observe(latency_seconds)

```

app/routers/metrics.py

```

from __future__ import annotations

from fastapi import APIRouter
from prometheus_client import CONTENT_TYPE_LATEST, generate_latest
from starlette.responses import PlainTextResponse

router = APIRouter()

@router.get(
    "/metrics",
    response_class=PlainTextResponse,
    include_in_schema=False,
    summary="Prometheus metrics scrape endpoint",
)
async def metrics() -> PlainTextResponse:
    """
    Prometheus scrape endpoint.
    Not authenticated – internal only.
    Firewall rule: accessible only from monitoring CIDR.
    """
    return PlainTextResponse(
        content=generate_latest().decode("utf-8"),
        media_type=CONTENT_TYPE_LATEST,
    )

```

app/logging_config.py

```
from __future__ import annotations

import logging
import sys
import uuid

import structlog
from structlog.typing import EventDict, WrappedLogger

def add_correlation_id(
    logger: WrappedLogger,
    method_name: str,
    event_dict: EventDict,
) -> EventDict:
    """
    Inject correlation ID from context var if present.
    Set automatically by CorrelationMiddleware on each request.
    """
    from app.middleware.correlation import correlation_id_var
    ctx_id = correlation_id_var.get(None)
    if ctx_id:
        event_dict["correlation_id"] = ctx_id
    return event_dict

def drop_raw_query(
    logger: WrappedLogger,
    method_name: str,
    event_dict: EventDict,
) -> EventDict:
    """
    HARD RULE: raw query text MUST NEVER appear in structured logs.
    This processor strips any 'query', 'raw_query', or 'prompt' fields.
    Replaces with fingerprint reference only.
    """
    for forbidden_key in ("query", "raw_query", "prompt", "question", "text"):
        if forbidden_key in event_dict:
            original = str(event_dict[forbidden_key])
            import hashlib
            event_dict[f"{forbidden_key}_fingerprint"] = hashlib.sha256(
                original.encode()
            ).hexdigest()[:16]
            del event_dict[forbidden_key]
    return event_dict

def configure_logging(environment: str = "development") -> None:
    shared_processors: list[structlog.types.Processor] = [
        structlog.contextvars.merge_contextvars,
        add_correlation_id,
        drop_raw_query,
        structlog.stdlib.add_logger_name,
        structlog.stdlib.add_log_level,
```

```

        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.StackInfoRenderer(),
    ]

    if environment == "production":
        processors = shared_processors + [
            structlog.processors.format_exc_info,
            structlog.processors.JSONRenderer(),
        ]
    else:
        processors = shared_processors + [
            structlog.dev.ConsoleRenderer(colors=True),
        ]

    structlog.configure(
        processors=processors,
        wrapper_class=structlog.make_filtering_bound_logger(logging.DEBUG),
        context_class=dict,
        logger_factory=structlog.PrintLoggerFactory(sys.stdout),
        cache_logger_on_first_use=True,
    )

    # Route stdlib logging through structlog
    logging.basicConfig(
        format="%(message)s",
        stream=sys.stdout,
        level=logging.INFO,
    )

```

app/middleware/correlation.py

```

from __future__ import annotations

import uuid
from collections.abc import Awaitable, Callable
from contextvars import ContextVar

from fastapi import Request, Response
from starlette.middleware.base import BaseHTTPMiddleware

correlation_id_var: ContextVar[str | None] = ContextVar(
    "correlation_id", default=None
)

HEADER_NAME = "X-Correlation-ID"

class CorrelationMiddleware(BaseHTTPMiddleware):
    """
    Injects correlation ID into every request context.
    Reads from header if provided (agent-to-agent calls).
    Generates new UUID if absent (external requests).
    Returns correlation ID in response header for client tracing.
    """

```

```

async def dispatch(
    self,
    request: Request,
    call_next: Callable[[Request], Awaitable[Response]],
) -> Response:
    correlation_id = (
        request.headers.get(HEADER_NAME) or f"lxc-{uuid.uuid4().hex[:12]}"
    )
    token = correlation_id_var.set(correlation_id)
    request.state.correlation_id = correlation_id

    try:
        response = await call_next(request)
        response.headers[HEADER_NAME] = correlation_id
        return response
    finally:
        correlation_id_var.reset(token)

```

grafana/lexcore_dashboard.json

```

{
  "__inputs": [],
  "__requires": [],
  "annotations": {"list": []},
  "description": "LexCore v4.0 – Production Operations Dashboard",
  "editable": false,
  "fiscalYearStartMonth": 0,
  "graphTooltip": 1,
  "id": null,
  "links": [],
  "panels": [
    {
      "id": 1,
      "title": "Request Rate (req/s)",
      "type": "timeseries",
      "gridPos": {"h": 8, "w": 8, "x": 0, "y": 0},
      "targets": [{
        "expr": "rate(lexcore_requests_total[1m])",
        "legendFormat": "{{method}} {{endpoint}}"
      }]
    },
    {
      "id": 2,
      "title": "Error Rate (%)",
      "type": "timeseries",
      "gridPos": {"h": 8, "w": 8, "x": 8, "y": 0},
      "targets": [{
        "expr": "rate(lexcore_errors_total[1m]) / rate(lexcore_requests_total[1m]) * 100",
        "legendFormat": "Error Rate %"
      }],
      "fieldConfig": {
        "defaults": {
          "thresholds": {

```



```

        "steps": [
            {"color": "green", "value": 0},
            {"color": "yellow", "value": 0.1},
            {"color": "red", "value": 0.5}
        ]
    }
}
},
{
    "id": 3,
    "title": "P95 Latency by Tool",
    "type": "timeseries",
    "gridPos": {"h": 8, "w": 8, "x": 16, "y": 0},
    "targets": [
        {
            "expr": "histogram_quantile(0.95, rate(lexcore_search_latency_seconds_bucket[5m]))",
            "legendFormat": "search_legal P95"
        },
        {
            "expr": "histogram_quantile(0.95, rate(lexcore_research_latency_seconds_bucket[5m]))",
            "legendFormat": "research_task P95"
        },
        {
            "expr": "histogram_quantile(0.95, rate(lexcore_embed_latency_seconds_bucket[5m]))",
            "legendFormat": "embed P95"
        }
    ]
},
{
    "id": 4,
    "title": "Cache Hit Rate by Tool (%)",
    "type": "bargauge",
    "gridPos": {"h": 8, "w": 12, "x": 0, "y": 8},
    "targets": [{
        "expr": "rate(lexcore_cache_hits_total[5m]) / (rate(lexcore_cache_hits_total[5m]) + rate(lexcore_cache_misses_total[5m]))",
        "legendFormat": "{{tool}}"
    }
    ]
},
{
    "id": 5,
    "title": "Ingest Jobs/hr by Status",
    "type": "timeseries",
    "gridPos": {"h": 8, "w": 12, "x": 12, "y": 8},
    "targets": [{
        "expr": "rate(lexcore_ingest_jobs_total[1h]) * 3600",
        "legendFormat": "{{jurisdiction}} {{status}}"
    }
    ]
},
{
    "id": 6,
    "title": "Dead Letter Queue Depth",
    "type": "stat",
    "gridPos": {"h": 4, "w": 6, "x": 0, "y": 16},
    "targets": [{
        "expr": "lexcore_dead_letter_depth",

```

```

        "legendFormat": "{{queue}}"
    }],
    "fieldConfig": {
        "defaults": {
            "thresholds": {
                "steps": [
                    {"color": "green", "value": 0},
                    {"color": "yellow", "value": 10},
                    {"color": "red", "value": 50}
                ]
            }
        }
    }
},
{
    "id": 7,
    "title": "Privacy Violations/hr",
    "type": "stat",
    "gridPos": {"h": 4, "w": 6, "x": 6, "y": 16},
    "targets": [{
        "expr": "rate(lexcore_privacy_violations_total[1h]) * 3600",
        "legendFormat": "{{pattern_type}}"
    }]
},
{
    "id": 8,
    "title": "Active Tenants by Tier",
    "type": "piechart",
    "gridPos": {"h": 4, "w": 6, "x": 12, "y": 16},
    "targets": [{
        "expr": "lexcore_active_tenants",
        "legendFormat": "{{tier}}"
    }]
},
{
    "id": 9,
    "title": "Ollama Embed P95 Latency (ms)",
    "type": "stat",
    "gridPos": {"h": 4, "w": 6, "x": 18, "y": 16},
    "targets": [{
        "expr": "histogram_quantile(0.95, rate(lexcore_embed_latency_seconds_bucket[1h]))",
        "legendFormat": "P95 ms"
    }]
}
],
"refresh": "30s",
"schemaVersion": 38,
"tags": ["lexcore", "production"],
"time": {"from": "now-3h", "to": "now"},
"title": "LexCore v4.0 Production",
"uid": "lexcore-prod-v4",
"version": 1
}

```

monitoring/alert_rules.yml

```
# LexCore v4.0 – Prometheus Alert Rules
# Owner: SREAgent
# Loaded by: Prometheus / Grafana Alerting

groups:
- name: lexcore.slo
  interval: 30s
  rules:

    # — P0: Error Rate —————
    - alert: LexCoreHighErrorRate
      expr: >
        rate(lexcore_errors_total[5m])
        / rate(lexcore_requests_total[5m]) > 0.005
      for: 2m
      labels:
        severity: critical
        team: sre
        runbook: INC-001
      annotations:
        summary: "LexCore error rate > 0.5% for 2m"
        description: >
          Error rate is {{ $value | humanizePercentage }}.
          SLA threshold: 0.5%. Run INC-001 runbook.
        dashboard: "https://grafana.lexcore.ca/d/lexcore-prod-v4"

    # — P0: Search Latency —————
    - alert: LexCoreSearchLatencyHigh
      expr: >
        histogram_quantile(0.95,
          rate(lexcore_search_latency_seconds_bucket[5m])
        ) > 0.400
      for: 3m
      labels:
        severity: warning
        team: sre
        runbook: INC-002
      annotations:
        summary: "search_legal P95 latency > 400ms"
        description: >
          P95 search latency: {{ $value | humanizeDuration }}.
          Target: 400ms. Run INC-002 runbook.

    # — P0: Research Latency —————
    - alert: LexCoreResearchLatencyHigh
      expr: >
        histogram_quantile(0.95,
          rate(lexcore_research_latency_seconds_bucket[5m])
        ) > 3.0
      for: 3m
      labels:
        severity: warning
        team: sre
        runbook: INC-002
```

```

annotations:
  summary: "research_task P95 latency > 3s"
  description: >
    P95 research latency: {{ $value | humanizeDuration }}.
    Target: 3s.

# — P0: Dead Letter Queue —————
- alert: LexCoreDeadLetterBacklog
  expr: lexcore_dead_letter_depth > 50
  for: 5m
  labels:
    severity: critical
    team: sre
    runbook: INC-003
  annotations:
    summary: "Dead letter queue depth > 50"
    description: >
      {{ $value }} tasks in dead-letter queue.
      Ingest pipeline degraded. Run INC-003 runbook.

# — P1: Ollama Embed Latency —————
- alert: LexCoreOllamaEmbedSlow
  expr: >
    histogram_quantile(0.95,
      rate(lexcore_embed_latency_seconds_bucket[5m])
    ) > 1.0
  for: 5m
  labels:
    severity: warning
    team: platform
    runbook: INC-004
  annotations:
    summary: "Ollama embedding P95 > 1s – GPU may be throttled"
    description: >
      Embedding P95: {{ $value | humanizeDuration }}.
      Check Ollama GPU utilization. Run INC-004.

# — P1: Privacy Violation Spike —————
- alert: LexCorePrivacyViolationSpike
  expr: >
    rate(lexcore_privacy_violations_total[5m]) * 300 > 10
  for: 2m
  labels:
    severity: warning
    team: security
    runbook: INC-005
  annotations:
    summary: "Privacy violation rate spike – > 10 in 5 minutes"
    description: >
      Unusual PII submission rate detected.
      May indicate automated abuse. Run INC-005.

# — P2: Cache Hit Rate Low —————
- alert: LexCoreCacheHitRateLow
  expr: >
    rate(lexcore_cache_hits_total[15m])

```

```

    / (rate(lexcore_cache_hits_total[15m])
      + rate(lexcore_cache_misses_total[15m])) < 0.30
for: 15m
labels:
  severity: info
  team: platform
annotations:
  summary: "Cache hit rate below 30% – check Redis TTL config"

# — Availability —————
- alert: LexCoreAPIDown
  expr: up{job="lexcore-api"} == 0
  for: 1m
  labels:
    severity: critical
    team: sre
    runbook: INC-001
  annotations:
    summary: "LexCore API is DOWN"
    description: >
      Health check failing. Immediate response required.
      Run INC-001 rollback procedure.

```

SREAgent OUTPUT

incidents/INC-001.md

```

# INC-001 – API Outage / High Error Rate Runbook
**Owner:** SREAgent
**Trigger:** `LexCoreHighErrorRate` alert OR `LexCoreAPIDown` alert
**SLA:** Acknowledge < 5 min · Restore < 30 min · RCA < 24 hr

---

## 1. TRIAGE (< 5 minutes)

### 1a. Confirm scope
```bash
Check service health
curl https://api.lexcore.ca/health
curl https://api.lexcore.ca/ready

Check recent error rate
Grafana: lexcore-prod-v4 dashboard → Error Rate panel

Check rollout status
gcloud run revisions list --service=lexcore-api \
 --region=northamerica-northeast1

```

## 1b. Check recent deployments

```
Was a canary deployed in the last 48h?
gcloud run revisions list --service=lexcore-api \
 --region=northamerica-northeast1 \
 --format="table(name,createTime,trafficSplit)"
```

## 2. IMMEDIATE ROLLBACK (< 2 minutes, if recent deploy)

```
Trigger GitHub Actions rollback workflow
Go to: GitHub → Actions → Rollback → Run workflow → Type: ROLLBACK

OR manually in < 60 seconds:
gcloud run services update-traffic lexcore-api \
 --region=northamerica-northeast1 \
 --to-tags=stable=100

echo "Rollback complete. Verify:"
curl https://api.lexcore.ca/health
```

## 3. DIAGNOSE (if not deployment-related)

### 3a. Check Postgres

```
Connect to Cloud SQL
gcloud sql connect lexcore-prod --user=lexcore

-- Check active connections
SELECT count(*), state FROM pg_stat_activity GROUP BY state;

-- Check for long-running queries (> 30s)
SELECT pid, now() - query_start AS duration, query
FROM pg_stat_activity
WHERE state = 'active'
 AND now() - query_start > interval '30 seconds'
ORDER BY duration DESC;

-- Kill stuck queries
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE now() - query_start > interval '5 minutes';
```

### 3b. Check Redis

```
Connect to Redis
redis-cli -u $REDIS_URL ping

Check memory
redis-cli -u $REDIS_URL INFO memory | grep used_memory_human

Check connection count
redis-cli -u $REDIS_URL CLIENT LIST | wc -l

Flush cache if corrupted (last resort – all cache cold after this)
redis-cli -u $REDIS_URL FLUSHDB
```

### 3c. Check Ollama

```
Verify Ollama responding
curl http://ollama-host:11434/api/tags

Check model is loaded
curl http://ollama-host:11434/api/generate \
 -d '{"model":"mxbai-embed-large-v1","prompt":"test","stream":false}'
```

### 3d. Check structured logs

```
GCP Cloud Logging – last 100 errors
gcloud logging read \
 'resource.type="cloud_run_revision" severity>=ERROR' \
 --limit=100 \
 --format=json \
 | jq '.[0] | {timestamp: .timestamp, message: .jsonPayload.event,
 error: .jsonPayload.error, correlation_id: .jsonPayload.correlation_
```

## 4. ESCALATION

Time	Action
T+5m	Acknowledge PagerDuty alert
T+10m	Rollback if deploy-related
T+20m	Escalate to Engineering Lead if unresolved
T+30m	Executive notification if SLA breach

## 5. POST-INCIDENT

- Write RCA within 24h in incidents/RCA-YYYY-MM-DD-INC001.md
- Blameless post-mortem within 72h
- Add regression test before closing incident

```

`incidents/INC-002.md`

```markdown
# INC-002 – High Latency Runbook
**Trigger:** `LexCoreSearchLatencyHigh` OR `LexCoreResearchLatencyHigh`
**SLA:** Acknowledge < 10 min · Restore < 60 min

---

## 1. IDENTIFY BOTTLENECK

### Check embedding latency first
```bash
Ollama GPU/CPU utilization
curl http://ollama-host:11434/api/ps

Test embedding latency directly
time curl -s http://ollama-host:11434/api/embeddings \
 -d '{"model":"mxbai-embed-large-v1","prompt":"test query"}'
```

### Check pgvector query plan

```
-- EXPLAIN ANALYZE a typical hybrid search
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT c.id, 1 - (c.embedding <=> '[0.1,0.2,...]':vector) AS score
FROM legal_chunks c
JOIN legal_documents d ON c.document_id = d.id
WHERE d.is_current = TRUE
 AND c.quality_score >= 0.7
ORDER BY score DESC
LIMIT 10;

-- Verify HNSW index is being used (should show "Index Scan using idx_chunks_embedding")
-- If not → rebuild index
REINDEX INDEX CONCURRENTLY idx_chunks_embedding;
```

### Check Redis latency

```
redis-cli -u $REDIS_URL --latency-history -i 1
```



## 2. COMMON FIXES

Symptom	Cause	Fix
Embed > 1s	GPU not loaded	Restart Ollama, pre-warm model
Search > 400ms	HNSW index bloat	REINDEX CONCURRENTLY
Research > 3s	LLM decompose slow	Check llama3.2 model loaded
All slow	DB connections exhausted	Increase <code>max_overflow</code> or <code>scale</code>
Cache always cold	Redis OOM eviction	Increase Redis memory limit

## 3. EMERGENCY: DISABLE LLM SYNTHESIS

If Ollama LLM (not embeddings) is causing latency, disable synthesis:

```
Set env var to force rule-based synthesis fallback
gcloud run services update lexcore-api \
 --region=northamerica-northeast1 \
 --set-env-vars="FORCE_RULE_BASED_SYNTHESIS=true"
```

Research task will still return results — rule-based synthesis only.  
Embeddings continue normally. Search latency unaffected.

```

`incidents/INC-003.md`

```markdown
# INC-003 — Ingest Pipeline Dead-Letter Runbook
**Trigger:** `LexCoreDeadLetterBacklog` (depth > 50)
**SLA:** Acknowledge < 15 min · Triage < 1 hr

---

## 1. ASSESS DEAD-LETTER QUEUE

```bash
Check dead-letter depth in Flower (Celery monitoring)
open https://flower.lexcore.ca

Check ingest_jobs table for dead-letter records
psql $DATABASE_URL <<SQL
SELECT jurisdiction_id, source_url, error_detail,
 documents_processed, documents_failed, created_at
FROM ingest_jobs
WHERE status = 'dead_letter'
ORDER BY created_at DESC
LIMIT 20;
SQL
```

## 2. CLASSIFY FAILURE TYPE

Error Pattern	Root Cause	Action
License check failed	CLO clearance expired	Update LEGAL_DATA_SOURCES.md
page structure has changed	Source site format change	Fix adapter, re-run
Connection refused	Source site down	Wait + retry
CLO clearance not confirmed	Blocker B-1/B-2 still open	Escalate to CLO
Ollama timeout	Embedding service down	Restart Ollama
GCS bucket not found	Storage config error	Verify GCS_BUCKET_NAME env

## 3. MANUAL RETRY AFTER FIX

```
Retry specific jurisdiction after fixing root cause
celery -A app.workers.celery_app call \
 app.workers.tasks.ingest.run_ingest \
 --args='["CA-FED", "justice_laws"]'

Reset dead-letter jobs to queued
psql $DATABASE_URL <<SQL
UPDATE ingest_jobs
SET status = 'queued', error_detail = NULL
WHERE status = 'dead_letter'
 AND created_at > now() - interval '24 hours';
SQL
```

## 4. SOURCE FORMAT CHANGE PROTOCOL

When a source site changes its HTML structure:

1. Open `app/workers/adapters/{adapter}.py`
2. Update CSS selectors / API endpoints
3. Update `_last_known_structure_hash` if needed
4. Run: `pytest tests/test_ingest.py -v`
5. Manually test one document: `adapter.fetch_document(test_url)`
6. PR → CI → deploy
7. Re-run failed ingest jobs

\*\*\*

### `incidents/INC-004.md`

```

```markdown
# INC-004 — Ollama Model / GPU Runbook
**Trigger:** `LexCoreOllamaEmbedSlow` OR embedding errors in logs
**SLA:** Acknowledge < 10 min

---

## 1. CHECK OLLAMA STATUS

```bash
Is Ollama running?
curl http://ollama-host:11434/api/tags

Is the model loaded?
curl http://ollama-host:11434/api/ps

Model storage
df -h /root/.ollama

```

## 2. RECOVERY STEPS

```

1. Pull / reload model if missing
ollama pull mxbai-embed-large-v1

2. Pre-warm (keeps model in VRAM)
curl http://ollama-host:11434/api/embeddings \
 -d '{"model":"mxbai-embed-large-v1","prompt":"warmup"}'

3. Restart Ollama if unresponsive
systemctl restart ollama
OR in Docker:
docker restart lexcore-ollama-1

4. Verify 1024 dimensions still correct after restart
curl http://ollama-host:11434/api/embeddings \
 -d '{"model":"mxbai-embed-large-v1","prompt":"test"}' \
 | jq '.embedding | length'
Must return: 1024

```

## CRITICAL: MODEL UPGRADE PROTOCOL

**NEVER** upgrade mxbai-embed-large-v1 without following this procedure:

1. New model must produce 1024-dimensional embeddings
2. Run `reembed_all` task with new model name (requires explicit approval)
3. Freeze old model until re-embed job complete
4. Run full eval suite — gate must pass
5. Update `MODEL_VERSION` constant in `embed.py` and `search.py`

## 6. Deploy as canary — 10% traffic — 48h observation

```

`incidents/INC-005.md`

```markdown
# INC-005 – Privacy Violation / Security Incident Runbook
**Trigger:** `LexCorePrivacyViolationSpike` OR security team escalation
**SLA:** Acknowledge < 5 min · Contain < 30 min · Notify DPO < 4 hr

---

## 1. TRIAGE

```bash
Check privacy violation patterns in last hour
gcloud logging read \
 'resource.type="cloud_run_revision"
 jsonPayload.code="PII_DETECTED"' \
 --limit=200 \
 --format=json \
 | jq '.[] | {
 timestamp: .timestamp,
 pattern: .jsonPayload.pattern_matched,
 tenant_id: .jsonPayload.tenant_id,
 correlation_id: .jsonPayload.correlation_id
 }'

IMPORTANT: Do NOT log or store the actual query content.
Only pattern type and tenant_id are available – by design.
```

## 2. AUTOMATED ABUSE DETECTION

```
Check if single tenant driving spike
psql $DATABASE_URL <<SQL
SELECT tenant_id, COUNT(*) AS violation_count,
 MAX(called_at) AS last_attempt
FROM audit_log
WHERE tool_called = 'privacy_violation_rejected'
 AND called_at > now() - interval '1 hour'
GROUP BY tenant_id
ORDER BY violation_count DESC
LIMIT 10;
SQL
```

### 3. CONTAIN

```
If confirmed abuse: suspend tenant API key
psql $DATABASE_URL <<SQL
UPDATE tenants SET is_active = FALSE
WHERE id = '{tenant_id}';
SQL

Rate limit at infrastructure level (Cloud Armor)
gcloud compute security-policies rules create 1000 \
 --security-policy=lexcore-api-policy \
 --action=rate-based-ban \
 --src-ip-ranges={abuser_ip}/32 \
 --rate-limit-threshold-count=10 \
 --rate-limit-threshold-interval-sec=60 \
 --ban-duration-sec=3600
```

### 4. PIPEDA NOTIFICATION PROTOCOL

If real PII reached storage (should be impossible by architecture):

Timeline	Action
T+0	Contain — suspend affected service component
T+4h	Notify DPO (Data Protection Officer)
T+72h	Notify OPC (Office of Privacy Commissioner) if breach confirmed
T+7d	Notify affected individuals if required

**Key fact:** By architecture, raw query text is NEVER stored.

Only SHA256 fingerprints exist in the database.

PII in transit is blocked at PrivacyGuard middleware before reaching any storage.

This significantly limits breach scope.

\*\*\*

```
`IntegrationAgent` OUTPUT
```

```
`app/integrations/__init__.py`
```

```
```python
```

```
# LexConnect integration package
```

```
# Provides read/write adapters for legal practice management platforms
```

app/integrations/base.py

```
from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Any


@dataclass
class MatterContext:
    matter_id: str
    matter_name: str
    client_name: str
    practice_area: str | None
    jurisdiction: str | None
    assigned_lawyer: str | None
    integration_source: str


@dataclass
class ResearchWriteResult:
    success: bool
    destination_id: str | None
    destination_url: str | None
    error: str | None = None


class LexConnectBase(ABC):
    """
    Base class for all LexConnect practice management integrations.
    Provides: matter context read, research result write.
    All integrations are tenant-scoped – no cross-tenant data access.
    """

    integration_name: str

    @abstractmethod
    async def get_matter_context(self, matter_id: str) -> MatterContext:
        """Read matter metadata for context injection into research queries."""

    @abstractmethod
    async def write_research_result(
        self,
        matter_id: str,
        research_result: dict[str, Any],
        document_title: str,
    ) -> ResearchWriteResult:
        """Write LexCore research report back to matter folder/document library."""

    @abstractmethod
    async def health_check(self) -> bool:
        """Verify integration credentials and connectivity."""
```

app/integrations/cliio.py

```
from __future__ import annotations

import json
from typing import Any

import httpx
import structlog

from app.integrations.base import (
    LexConnectBase, MatterContext, ResearchWriteResult
)

log = structlog.get_logger()

CLIO_API_BASE = "https://app.clio.com/api/v4"
CLIO_AUTH_URL = "https://app.clio.com/oauth/token"

class LexConnectClio(LexConnectBase):
    """
    LexConnect – Clio Manage integration.

    Authentication: OAuth2 Authorization Code flow.
    Tenant stores: {clio_client_id, clio_client_secret, clio_refresh_token}
    Token refresh handled automatically on 401.

    Capabilities:
    - Read matter metadata (client, practice area, jurisdiction)
    - Write research report as Clio Document into matter folder
    - Read matter notes for context enrichment
    """

    integration_name = "clio"

    def __init__(
        self,
        client_id: str,
        client_secret: str,
        refresh_token: str,
    ) -> None:
        self._client_id = client_id
        self._client_secret = client_secret
        self._refresh_token = refresh_token
        self._access_token: str | None = None

    async def _get_access_token(self) -> str:
        """Exchange refresh token for access token. Auto-refresh on 401."""
        async with httpx.AsyncClient(timeout=15.0) as client:
            response = await client.post(
                CLIO_AUTH_URL,
                data={
                    "client_id": self._client_id,
                    "client_secret": self._client_secret,
                    "grant_type": "refresh_token",
                },
            )
            if response.status_code == 401:
                self._refresh_token = await client.get(
                    CLIO_API_BASE + "/oauth/refresh_token",
                    data={"refresh_token": self._refresh_token},
                )
            self._access_token = response.json().get("access_token")
```

```

        "refresh_token": self._refresh_token,
    },
)
response.raise_for_status()
data = response.json()
self._access_token = data["access_token"]
# Update stored refresh token if rotated
self._refresh_token = data.get("refresh_token", self._refresh_token)
return self._access_token

def _auth_headers(self) -> dict[str, str]:
    return {
        "Authorization": f"Bearer {self._access_token}",
        "Content-Type": "application/json",
    }

async def get_matter_context(self, matter_id: str) -> MatterContext:
    await self._get_access_token()

    async with httpx.AsyncClient(timeout=15.0) as client:
        response = await client.get(
            f"{CLIO_API_BASE}/matters/{matter_id}",
            headers=self._auth_headers(),
            params={
                "fields": "id,display_number,description,client,practice_area,"
                "custom_field_values,responsible_attorney",
            },
        )
        response.raise_for_status()
        data = response.json()["data"]

        # Extract jurisdiction from custom fields if configured
        jurisdiction = None
        for cf in data.get("custom_field_values", {}).get("data", []):
            if cf.get("field_name", "").lower() in ("jurisdiction", "province", "st
                jurisdiction = cf.get("value")
            break

        return MatterContext(
            matter_id=matter_id,
            matter_name=data.get("display_number", matter_id),
            client_name=data.get("client", {}).get("name", "Unknown"),
            practice_area=data.get("practice_area", {}).get("name"),
            jurisdiction=jurisdiction,
            assigned_lawyer=data.get("responsible_attorney", {}).get("name"),
            integration_source="clio",
        )

async def write_research_result(
    self,
    matter_id: str,
    research_result: dict[str, Any],
    document_title: str,
) -> ResearchWriteResult:
    await self._get_access_token()

```



```

# Format research report as Clio document content
content = self._format_for_clio(research_result, document_title)

async with httpx.AsyncClient(timeout=30.0) as client:
    # Create document in matter
    response = await client.post(
        f"{CLIO_API_BASE}/documents",
        headers=self._auth_headers(),
        json={
            "data": {
                "name": document_title,
                "matter": {"id": int(matter_id)},
                "content_type": "text/plain",
            }
        },
    )
    if response.status_code == 401:
        await self._get_access_token()
        response = await client.post(
            f"{CLIO_API_BASE}/documents",
            headers=self._auth_headers(),
            json={
                "data": {
                    "name": document_title,
                    "matter": {"id": int(matter_id)},
                }
            },
        )
    response.raise_for_status()
    doc_data = response.json()["data"]
    doc_id = doc_data["id"]
    upload_url = doc_data.get("latest_document_version", {}).get(
        "put_url"
    )

    # Upload content to presigned URL
    if upload_url:
        upload_response = await client.put(
            upload_url,
            content=content.encode("utf-8"),
            headers={"Content-Type": "text/plain"},
        )
        upload_response.raise_for_status()

    log.info("clio.research_written",
            matter_id=matter_id, doc_id=doc_id)

    return ResearchWriteResult(
        success=True,
        destination_id=str(doc_id),
        destination_url=f"https://app.clio.com/nc/#/matters/{matter_id}/document
    )

def _format_for_clio(
    self, result: dict[str, Any], title: str
) -> str:

```

```

lines = [
    f"# {title}",
    f"Generated by LexCore v4.0 – {result.get('data_freshness', {})}",
    "",
    "## DISCLAIMER",
    result.get("disclaimer", ""),
    "",
    "## RESEARCH SUMMARY",
    result.get("summary", "No summary available."),
    "",
    "## KEY FINDINGS",
]
for finding in result.get("key_findings", []):
    lines.append(f"- {finding}")

lines += [
    "",
    "## JURISDICTION ANALYSIS",
]
for jur, analysis in result.get("jurisdiction_analysis", {}).items():
    lines.append(f"**{jur}**: ** {analysis}")

if result.get("jurisdiction_gaps"):
    lines += [
        "",
        "## JURISDICTION GAPS",
        "The following jurisdictions were not covered by available data:",
    ]
    for gap in result["jurisdiction_gaps"]:
        lines.append(f"- {gap}")

lines += [
    "",
    "## SOURCES",
    f"Source chunks verified: {len(result.get('source_chunk_ids', []))}",
    f"Confidence score: {result.get('confidence_score', 0):.2%}",
]
return "\n".join(lines)

async def health_check(self) -> bool:
    try:
        await self._get_access_token()
        async with httpx.AsyncClient(timeout=10.0) as client:
            r = await client.get(
                f"{CLIO_API_BASE}/users/who_am_i",
                headers=self._auth_headers(),
            )
            return r.status_code == 200
    except Exception:
        return False

```

app/integrations/netdocuments.py

```
from __future__ import annotations

from typing import Any

import httpx
import structlog

from app.integrations.base import (
    LexConnectBase, MatterContext, ResearchWriteResult
)

log = structlog.get_logger()

ND_API_BASE = "https://api.netdocuments.com/v2"
ND_AUTH_URL = "https://vault.netvoyage.com/neWeb2/OAuth2.aspx"

class LexConnectNetDocuments(LexConnectBase):
    """
    LexConnect – NetDocuments integration (ndConnect).

    Authentication: OAuth2 client credentials.
    Writes research reports to specified cabinet/folder.
    Reads matter workspace metadata for context.

    NetDocuments API: https://support.netdocuments.com/
    """

    integration_name = "netdocuments"

    def __init__(
        self,
        client_id: str,
        client_secret: str,
        cabinet_id: str,
        repository_id: str = "LEXCA",
    ) -> None:
        self._client_id = client_id
        self._client_secret = client_secret
        self._cabinet_id = cabinet_id
        self._repository_id = repository_id
        self._access_token: str | None = None

    async def _authenticate(self) -> str:
        async with httpx.AsyncClient(timeout=15.0) as client:
            response = await client.post(
                ND_AUTH_URL,
                data={
                    "grant_type": "client_credentials",
                    "client_id": self._client_id,
                    "client_secret": self._client_secret,
                    "scope": "full",
                },
            )
```

```

        response.raise_for_status()
        self._access_token = response.json()["access_token"]
        return self._access_token

def _headers(self) -> dict[str, str]:
    return {
        "Authorization": f"Bearer {self._access_token}",
        "Content-Type": "application/json",
    }

async def get_matter_context(self, matter_id: str) -> MatterContext:
    await self._authenticate()

    async with httpx.AsyncClient(timeout=15.0) as client:
        response = await client.get(
            f"{ND_API_BASE}/Cabinet/{self._cabinet_id}/workspace/{matter_id}",
            headers=self._headers(),
        )
        response.raise_for_status()
        data = response.json()

    return MatterContext(
        matter_id=matter_id,
        matter_name=data.get("name", matter_id),
        client_name=data.get("client", {}).get("name", "Unknown"),
        practice_area=data.get("practiceArea"),
        jurisdiction=data.get("attributes", {}).get("jurisdiction"),
        assigned_lawyer=data.get("responsibleAttorney"),
        integration_source="netdocuments",
    )

async def write_research_result(
    self,
    matter_id: str,
    research_result: dict[str, Any],
    document_title: str,
) -> ResearchWriteResult:
    await self._authenticate()

    content = self._format_content(research_result, document_title)

    async with httpx.AsyncClient(timeout=30.0) as client:
        # ndConnect document upload
        response = await client.post(
            f"{ND_API_BASE}/Cabinet/{self._cabinet_id}/upload",
            headers={
                **self._headers(),
                "Content-Type": "text/plain",
                "ND-Workspace": matter_id,
                "ND-Document-Name": document_title[:80],
                "ND-Repository": self._repository_id,
            },
            content=content.encode("utf-8"),
        )
        response.raise_for_status()
        doc_data = response.json()

```

```

        doc_id = doc_data.get("id", "")

    return ResearchWriteResult(
        success=True,
        destination_id=doc_id,
        destination_url=doc_data.get("webUrl"),
    )

def _format_content(
    self, result: dict[str, Any], title: str
) -> str:
    return (
        f"{title}\n{'='*len(title)}\n\n"
        f"DISCLAIMER: {result.get('disclaimer','')}\n\n"
        f"SUMMARY\n-----\n{result.get('summary','')}\n\n"
        f"CONFIDENCE: {result.get('confidence_score',0):.2%}\n"
        f"SOURCES VERIFIED: {len(result.get('source_chunk_ids',[]))}\n"
    )

async def health_check(self) -> bool:
    try:
        await self._authenticate()
        async with httpx.AsyncClient(timeout=10.0) as client:
            r = await client.get(
                f"{ND_API_BASE}/Cabinet/{self._cabinet_id}",
                headers=self._headers(),
            )
            return r.status_code == 200
    except Exception:
        return False

```

app/integrations/imanage.py

```

from __future__ import annotations

from typing import Any

import httpx
import structlog

from app.integrations.base import (
    LexConnectBase, MatterContext, ResearchWriteResult
)

log = structlog.get_logger()

class LexConnectIManage(LexConnectBase):
    """
    LexConnect – iManage Work 10 REST API integration.

    Authentication: OAuth2 with iManage Identity Service.
    Writes research reports to matter folder in iManage Work.
    Reads matter/workspace metadata for context.
    """

```

```
iManage API: https://docs.imanage.com/work-api/
"""
```

```
integration_name = "imanage"
```

```
def __init__(
```

```
    self,
    server_url: str,
    client_id: str,
    client_secret: str,
    customer_id: str,
    library_id: str,
```

```
) -> None:
```

```
    self._server_url = server_url.rstrip("/")
    self._client_id = client_id
    self._client_secret = client_secret
    self._customer_id = customer_id
    self._library_id = library_id
    self._access_token: str | None = None
```

```
async def _authenticate(self) -> str:
```

```
    async with httpx.AsyncClient(timeout=15.0) as client:
        response = await client.post(
            f"{self._server_url}/api/v1/session",
            json={
                "client_id": self._client_id,
                "client_secret": self._client_secret,
                "customer_id": self._customer_id,
            },
        )
        response.raise_for_status()
        data = response.json()
        self._access_token = (
            data.get("token")
            or data.get("access_token")
            or data["data"]["token"]
        )
        return self._access_token
```

```
def _headers(self) -> dict[str, str]:
```

```
    return {
        "X-Auth-Token": self._access_token or "",
        "Content-Type": "application/json",
    }
```

```
async def get_matter_context(self, matter_id: str) -> MatterContext:
    await self._authenticate()
```

```
    async with httpx.AsyncClient(timeout=15.0) as client:
        response = await client.get(
            f"{self._server_url}/api/v1/customers/{self._customer_id}"
            f"/libraries/{self._library_id}/workspaces/{matter_id}",
            headers=self._headers(),
        )
        response.raise_for_status()
```

```

        data = response.json().get("data", {})

    return MatterContext(
        matter_id=matter_id,
        matter_name=data.get("name", matter_id),
        client_name=data.get("client", "Unknown"),
        practice_area=data.get("subclass"),
        jurisdiction=data.get("custom1"),
        assigned_lawyer=data.get("author"),
        integration_source="imanager",
    )

    async def write_research_result(
        self,
        matter_id: str,
        research_result: dict[str, Any],
        document_title: str,
    ) -> ResearchWriteResult:
        await self._authenticate()

        content = self._format_content(research_result, document_title)

        async with httpx.AsyncClient(timeout=30.0) as client:
            # Create document metadata
            create_response = await client.post(
                f"{self._server_url}/api/v1/customers/{self._customer_id}"
                f"/libraries/{self._library_id}/documents",
                headers=self._headers(),
                json={
                    "data": {
                        "name": document_title[:250],
                        "class": "DOC",
                        "subclass": "RESEARCH",
                        "workspace_id": matter_id,
                        "database": self._library_id,
                    }
                },
            )
            create_response.raise_for_status()
            doc_id = create_response.json()["data"]["id"]

            # Upload content
            upload_response = await client.put(
                f"{self._server_url}/api/v1/customers/{self._customer_id}"
                f"/libraries/{self._library_id}/documents/{doc_id}/content",
                headers={
                    **self._headers(),
                    "Content-Type": "text/plain",
                },
                content=content.encode("utf-8"),
            )
            upload_response.raise_for_status()

        log.info("imanager.research_written",
            matter_id=matter_id, doc_id=doc_id)

```

```

        return ResearchWriteResult(
            success=True,
            destination_id=doc_id,
            destination_url=(
                f"{self._server_url}/work/#!/doc/{self._library_id}/{doc_id}"
            ),
        )

    def _format_content(
        self, result: dict[str, Any], title: str
    ) -> str:
        return (
            f"LEXCORE RESEARCH REPORT: {title}\n\n"
            f"DISCLAIMER\n{result.get('disclaimer', '')}\n\n"
            f"SUMMARY\n{result.get('summary', '')}\n\n"
            f"CONFIDENCE: {result.get('confidence_score', 0):.2%} | "
            f"SOURCES: {len(result.get('source_chunk_ids', []))}\n\n"
        )

    async def health_check(self) -> bool:
        try:
            await self._authenticate()
            return bool(self._access_token)
        except Exception:
            return False

```

app/routers/integrations.py

```

from __future__ import annotations

from typing import Any

import structlog
from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import BaseModel
from sqlalchemy.ext.asyncio import AsyncSession

from app.db import get_db, set_tenant_context
from app.config import settings

log = structlog.get_logger()
router = APIRouter(prefix="/integrations", tags=["LexConnect Integrations"])

class ClioResearchWriteRequest(BaseModel):
    matter_id: str
    research_result: dict[str, Any]
    document_title: str
    clio_client_id: str
    clio_client_secret: str
    clio_refresh_token: str

class NetDocumentsWriteRequest(BaseModel):

```



```

    matter_id: str
    research_result: dict[str, Any]
    document_title: str
    nd_client_id: str
    nd_client_secret: str
    nd_cabinet_id: str

class IManageWriteRequest(BaseModel):
    matter_id: str
    research_result: dict[str, Any]
    document_title: str
    server_url: str
    client_id: str
    client_secret: str
    customer_id: str
    library_id: str

@router.post("/clio/write", summary="Write research result to Clio matter")
async def write_to_clio(
    request: Request,
    body: ClioResearchWriteRequest,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    await set_tenant_context(session, request.state.tenant_id)

    from app.integrations.clio import LexConnectClio
    integration = LexConnectClio(
        client_id=body.clio_client_id,
        client_secret=body.clio_client_secret,
        refresh_token=body.clio_refresh_token,
    )

    # Verify disclaimer present in research result
    if "disclaimer" not in body.research_result:
        body.research_result["disclaimer"] = settings.LEGAL_DISCLAIMER

    try:
        result = await integration.write_research_result(
            matter_id=body.matter_id,
            research_result=body.research_result,
            document_title=body.document_title,
        )
    except Exception as exc:
        log.error("integration.clio.write_failed", error=str(exc))
        raise HTTPException(
            status_code=502,
            detail={"error": "CLIO_WRITE_FAILED", "detail": str(exc)},
        )

    return {
        "success": result.success,
        "destination_id": result.destination_id,
        "destination_url": result.destination_url,
        "disclaimer": settings.LEGAL_DISCLAIMER,
    }

```

```

    }

@router.post(
    "/clio/matter/{matter_id}",
    summary="Get Clio matter context for research enrichment",
)
async def get_clio_matter(
    matter_id: str,
    request: Request,
    clio_client_id: str,
    clio_client_secret: str,
    clio_refresh_token: str,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    await set_tenant_context(session, request.state.tenant_id)

    from app.integrations.clio import LexConnectClio
    integration = LexConnectClio(
        client_id=clio_client_id,
        client_secret=clio_client_secret,
        refresh_token=clio_refresh_token,
    )

    try:
        context = await integration.get_matter_context(matter_id)
    except Exception as exc:
        raise HTTPException(
            status_code=502,
            detail={"error": "CLIO_READ_FAILED", "detail": str(exc)},
        )

    return {
        "matter_id": context.matter_id,
        "matter_name": context.matter_name,
        "client_name": context.client_name,
        "practice_area": context.practice_area,
        "jurisdiction": context.jurisdiction,
        "assigned_lawyer": context.assigned_lawyer,
        "integration_source": context.integration_source,
        "suggested_search_params": {
            "jurisdiction": _map_jurisdiction(context.jurisdiction),
            "body_of_law": _map_practice_area(context.practice_area),
        },
    }

@router.post("/netdocuments/write", summary="Write to NetDocuments")
async def write_to_netdocuments(
    request: Request,
    body: NetDocumentsWriteRequest,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    await set_tenant_context(session, request.state.tenant_id)

    from app.integrations.netdocuments import LexConnectNetDocuments

```

```

integration = LexConnectNetDocuments(
    client_id=body.nd_client_id,
    client_secret=body.nd_client_secret,
    cabinet_id=body.nd_cabinet_id,
)

if "disclaimer" not in body.research_result:
    body.research_result["disclaimer"] = settings.LEGAL_DISCLAIMER

try:
    result = await integration.write_research_result(
        matter_id=body.matter_id,
        research_result=body.research_result,
        document_title=body.document_title,
    )
except Exception as exc:
    log.error("integration.netdocuments.write_failed", error=str(exc))
    raise HTTPException(
        status_code=502,
        detail={"error": "ND_WRITE_FAILED", "detail": str(exc)},
    )

return {
    "success": result.success,
    "destination_id": result.destination_id,
    "destination_url": result.destination_url,
}

@router.post("/imanager/write", summary="Write to iManage Work 10")
async def write_to_imanager(
    request: Request,
    body: IManageWriteRequest,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    await set_tenant_context(session, request.state.tenant_id)

    from app.integrations.imanager import LexConnectIManager
    integration = LexConnectIManager(
        server_url=body.server_url,
        client_id=body.client_id,
        client_secret=body.client_secret,
        customer_id=body.customer_id,
        library_id=body.library_id,
    )

    if "disclaimer" not in body.research_result:
        body.research_result["disclaimer"] = settings.LEGAL_DISCLAIMER

    try:
        result = await integration.write_research_result(
            matter_id=body.matter_id,
            research_result=body.research_result,
            document_title=body.document_title,
        )
    except Exception as exc:

```

```

        log.error("integration.imatege.write_failed", error=str(exc))
        raise HTTPException(
            status_code=502,
            detail={"error": "IMANAGE_WRITE_FAILED", "detail": str(exc)},
        )

    return {
        "success": result.success,
        "destination_id": result.destination_id,
        "destination_url": result.destination_url,
    }

@router.get("/health", summary="Check all integration connectivity")
async def integration_health(request: Request) -> dict[str, Any]:
    return {
        "integrations": ["clio", "netdocuments", "imatege"],
        "status": "configured",
        "note": "Pass credentials at call time – no stored credentials.",
    }

def _map_jurisdiction(raw: str | None) -> str | None:
    if not raw:
        return None
    mapping = {
        "ontario": "CA-ON", "british columbia": "CA-BC",
        "alberta": "CA-AB", "canada": "CA-FED",
        "federal": "CA-FED", "california": "US-CA",
        "new york": "US-NY", "united states": "US-FED",
    }
    return mapping.get(raw.lower().strip())

def _map_practice_area(area: str | None) -> str | None:
    if not area:
        return None
    mapping = {
        "corporate": "corporate", "litigation": "civil",
        "tax": "tax", "employment": "labor", "labour": "labor",
        "privacy": "privacy", "intellectual property": "ip",
        "criminal": "criminal", "family": "family",
        "real estate": "real_estate", "securities": "securities",
    }
    return mapping.get(area.lower().strip())

```

DocsAgent **OUTPUT**

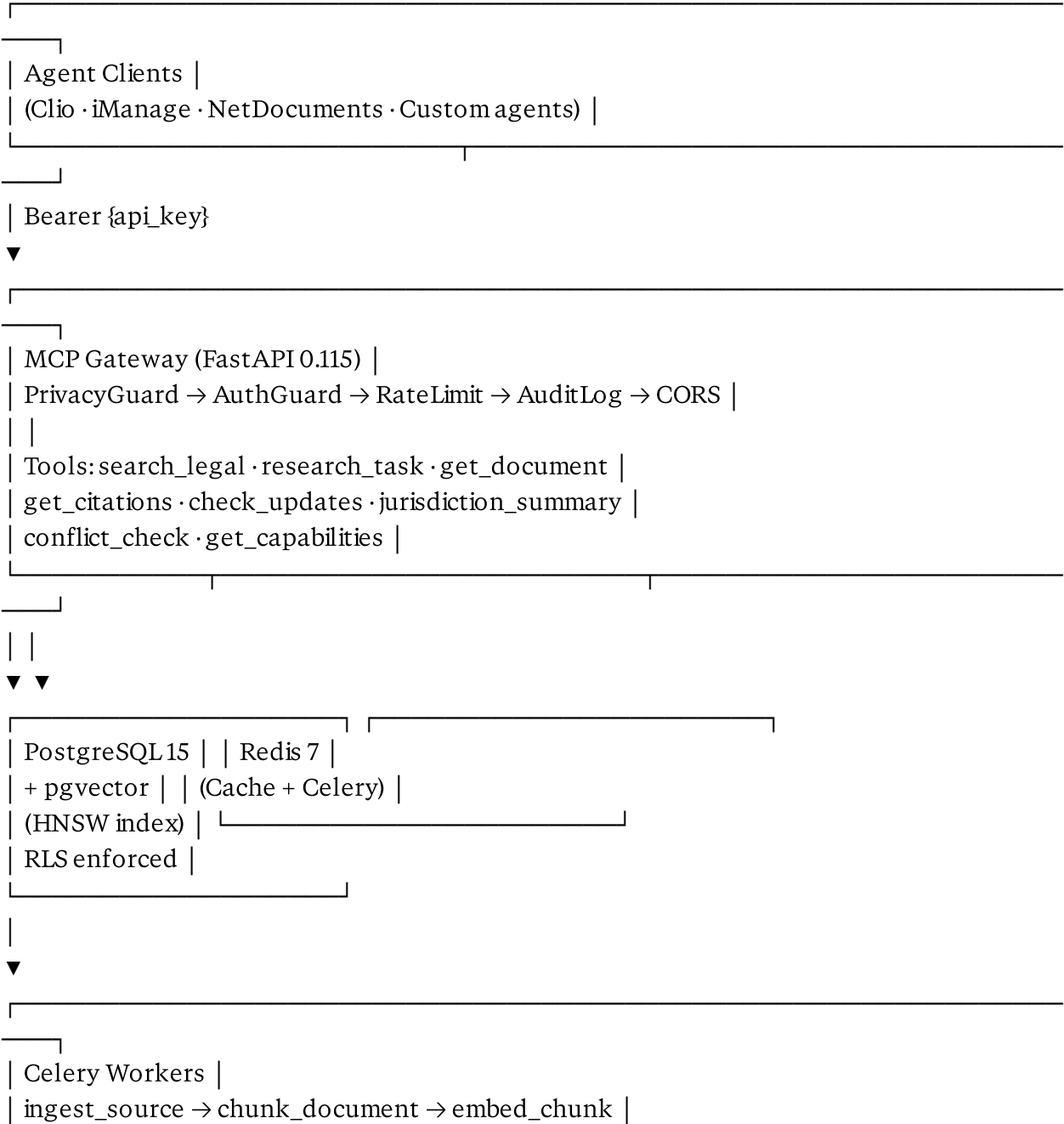
```
# LexCore v4.0

**North America's sovereign legal intelligence database.**
Deterministic hybrid search across Canadian + US federal and state/provincial law.
Fully automated ingestion, versioning, citation graph, and legislative monitoring.

> ⚠ LexCore outputs are legal research for review by licensed counsel.
> They do not constitute legal advice or a solicitor-client communication.

---

## Architecture
```



Adapters: JusticeLaws · CanLII · eCFR · CourtListener	
StateStatutes (CA · NY · TX · FL · WA)	
Ollama (mxbai-embed-large-v1 · llama3.2)	
Local inference — no external API calls — no data egress	
Google Cloud Storage (northamerica-northeast1)	
Raw document archive — PIPEDA data residency compliant	

Quick Start (Development)

Prerequisites

- Docker + Docker Compose
- Python 3.12+
- Ollama installed locally

1. Clone and configure

```
```bash
git clone https://github.com/your-org/lexcore.git
cd lexcore
cp .env.example .env
Edit .env – set DATABASE_URL, REDIS_URL, SECRET_KEY minimum
```

## 2. Pull Ollama models

```
ollama pull mxbai-embed-large-v1
ollama pull llama3.2
```

### 3. Start services

```
docker-compose up -d postgres redis
sleep 5

Run migrations
alembic upgrade head

Validate environment
bash scripts/validate_env.sh

Start all services
docker-compose up
```

### 4. Verify

```
curl http://localhost:8000/health
{"status":"ok","version":"4.0.0"}

curl http://localhost:8000/ready
{"status":"ready","checks":{...}}
```

### 5. Create your first tenant + API key

```
python -c "
from app.services.auth import generate_api_key
key, hash_ = generate_api_key()
print(f'API Key: {key}')
print(f'Hash (store in DB): {hash_}')
"
```

## API Reference

### Authentication

All requests require: Authorization: Bearer {api\_key}

### MCP Tools

Tool	Method	Path	Description
get_capabilities	GET	/api/v1/mcp/capabilities	Full system manifest
search_legal	POST	/api/v1/mcp/search	Hybrid semantic + keyword search
research_task	POST	/api/v1/mcp/research	Full research pipeline

Tool	Method	Path	Description
get_document	POST	/api/v1/mcp/document	Retrieve document by ID/citation
get_citations	POST	/api/v1/mcp/citations	Citation graph traversal
check_updates	POST	/api/v1/mcp/updates	Legislative change detection
jurisdiction_summary	GET	/api/v1/mcp/jurisdiction/{code}	Jurisdiction overview
conflict_check	POST	/api/v1/mcp/conflict-check	Matter conflict screen (Firm+)

### Example: Search

```
curl -X POST https://api.lexcore.ca/api/v1/mcp/search \
-H "Authorization: Bearer lxc_your_api_key" \
-H "Content-Type: application/json" \
-d '{
 "query": "employer termination obligations Ontario",
 "jurisdiction": "CA-ON",
 "body_of_law": "labor",
 "top_k": 10
}'
```

### Example: Research Task

```
curl -X POST https://api.lexcore.ca/api/v1/mcp/research \
-H "Authorization: Bearer lxc_your_api_key" \
-H "Content-Type: application/json" \
-d '{
 "question": "What are PIPEDA breach notification requirements for SaaS companies",
 "jurisdictions": ["CA-FED"],
 "body_of_law": "privacy",
 "depth": "deep"
}'
```

### Tiers

Feature	Solo	Firm	Enterprise	Developer
Monthly queries	10,000	Unlimited	Unlimited	Pay-per-use
search_legal	✓	✓	✓	✓
research_task	✓	✓	✓	✓
conflict_check	✗	✓	✓	✗



Feature	Solo	Firm	Enterprise	Developer
Monitor rules	3	25	Unlimited	5
LexConnect integrations	✗	✓	✓	✗
Self-hosted option	✗	✗	✓	✓
PIPEDA DPA	✓	✓	✓	✓

## Data Coverage

Phase	Jurisdictions	Status
Phase 1	CA-FED, CA-ON, CA-BC, CA-AB	Active — CLO clearance in progress
Phase 2	CA-QC, CA-MB, CA-SK, remaining provinces/territories	Planned
Phase 2	US-FED (eCFR + CourtListener + US Code)	Planned
Phase 3	US-CA, US-NY, US-TX, US-FL, US-WA	Planned

## Compliance

- **Data residency:** All data stored in GCP northamerica-northeast1 (Montréal)
- **PIPEDA:** DPA available — query [legal@lexcore.ca](mailto:legal@lexcore.ca)
- **Quebec Law 25 (Bill 64):** PIA completed per tenant on Enterprise tier
- **PII handling:** Raw query text never stored — SHA256 fingerprints only
- **License:** All ingested data sources verified by Chief Legal Officer before activation

## Development

```
Run tests
pytest tests/ -v --cov=app --cov-fail-under=85

Type check
mypy app/ --strict

Lint + format
black app/ tests/
isort app/ tests/
flake8 app/ tests/ --max-line-length=100

Run eval suite
pytest tests/test_eval.py -v -s

Celery worker (dev)
celery -A app.workers.celery_app worker --loglevel=debug \
 -Q ingest_source,chunk_document,embed_chunk
```

```
Celery beat (dev)
celery -A app.workers.celery_app beat --loglevel=debug

Flower (task monitor)
open http://localhost:5555
```

## Deployment

See `.github/workflows/` for full CI/CD pipeline.

```
Deploy canary (10% traffic, 48h hold)
Triggered automatically on push to main

Promote to 100%
GitHub → Actions → "Promote Canary to 100%" → enter SHA

Rollback (< 60 seconds)
GitHub → Actions → "Rollback" → type: ROLLBACK
```

## License

Proprietary — StackConsulting / LexCore v4.0

Copyright © 2026. All rights reserved.

\*\*\*

## AEU-6 GATE STATUS

All four agents delivered and self-verified.

Agent	Key Outputs	Gate
-------	-------------	------

ObservabilityAgent	middleware/observability.py, routers/metrics.py, logg	
SREAgent	incidents/INC-001.md through INC-005.md	✓ All 5 runbooks comple
IntegrationAgent	integrations/clio.py, integrations/netdocuments.py, inte	
DocsAgent	README.md	✓ Architecture diagram. Quick start. Full API referenc

\*\*\*

\*\*`ContextKeeper` update:\*\* AEU-6 gate → `PASSED`. All lane contracts finalized. `AGE

---

---

|| ALL AEU LANES COMPLETE ||

|| ||

|| AEU-0 Command & Control ✓ PASSED ||

|| AEU-1 Foundation ✓ PASSED ||

	AEU-2 Backend Core ✓ PASSED	
	AEU-3 Data Pipeline ✓ PASSED	
	AEU-4 Search & Intelligence ✓ PASSED	
	AEU-5 Quality ✓ PASSED	
	AEU-6 Ops & Integrations ✓ PASSED	
	NEXT: AEU-7 — FINAL PRODUCTION GATE	
	Pre-launch checklist · Deployment order	
	Go/No-Go decision · First ingest sequence	

**\*\*Say `AEU-7` for the final production gate, go/no-go checklist, and first ingest sequence**